```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import linear_model
```

LOODING THE DATASET

```
dataset=pd.read_csv('/content/housing.csv')
```

To Show the Data set

```
dataset.head()
```

```
   longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
0    -122.23     37.88                  41          880
129.0
1    -122.22     37.86                  21         7099
1106.0
2    -122.24     37.85                  52         1467
190.0
3    -122.25     37.85                  52         1274
235.0
4    -122.25     37.85                  52         1627
280.0

   population  households  median_income  median_house_value
ocean_proximity
0         322         126         8.3252              452600
NEAR BAY
1        2401        1138         8.3014              358500
NEAR BAY
2         496         177         7.2574              352100
NEAR BAY
3         558         219         5.6431              341300
NEAR BAY
4         565         259         3.8462              342200
NEAR BAY
```

to show the data information, types

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  int64
 3   total_rooms         20640 non-null  int64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  int64
 6   households          20640 non-null  int64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  int64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(4), int64(5), object(1)
memory usage: 1.6+ MB
```

DATA EXPLORATION

To Check the Null Value

```
dataset.dropna(inplace=True)

dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 20433 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20433 non-null  float64
 1   latitude            20433 non-null  float64
 2   housing_median_age  20433 non-null  int64
 3   total_rooms         20433 non-null  int64
 4   total_bedrooms      20433 non-null  float64
 5   population          20433 non-null  int64
 6   households          20433 non-null  int64
 7   median_income       20433 non-null  float64
 8   median_house_value  20433 non-null  int64
 9   ocean_proximity     20433 non-null  object
dtypes: float64(4), int64(5), object(1)
memory usage: 1.7+ MB
```

To split the dataset into x (for train data) and y (for test data)

```
from sklearn.model_selection import train_test_split
x=dataset.drop(['median_house_value'], axis=1)
y=dataset['median_house_value']

x
```

```
       longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
0        -122.23     37.88                  41          880
129.0
1        -122.22     37.86                  21         7099
1106.0
2        -122.24     37.85                  52         1467
190.0
3        -122.25     37.85                  52         1274
235.0
4        -122.25     37.85                  52         1627
280.0
...          ...       ...                 ...          ...
...
20635    -121.09     39.48                  25         1665
374.0
20636    -121.21     39.49                  18          697
150.0
20637    -121.22     39.43                  17         2254
485.0
20638    -121.32     39.43                  18         1860
409.0
20639    -121.24     39.37                  16         2785
616.0

       population  households  median_income ocean_proximity
0             322         126         8.3252        NEAR BAY
1            2401        1138         8.3014        NEAR BAY
2             496         177         7.2574        NEAR BAY
3             558         219         5.6431        NEAR BAY
4             565         259         3.8462        NEAR BAY
...           ...         ...            ...             ...
20635         845         330         1.5603          INLAND
20636         356         114         2.5568          INLAND
20637        1007         433         1.7000          INLAND
20638         741         349         1.8672          INLAND
20639        1387         530         2.3886          INLAND

[20433 rows x 9 columns]
```

```
y
```

```
0        452600
1        358500
2        352100
3        341300
4        342200
          ...
20635     78100
20636     77100
20637     92300
20638     84700
20639     89400
Name: median_house_value, Length: 20433, dtype: int64
```

```
x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.2)
```

We join x training data into y training data for analysis with basic corelation

```
train_data=x_train.join(y_train)
```

now we shaw the combined train data

```
train_data
```

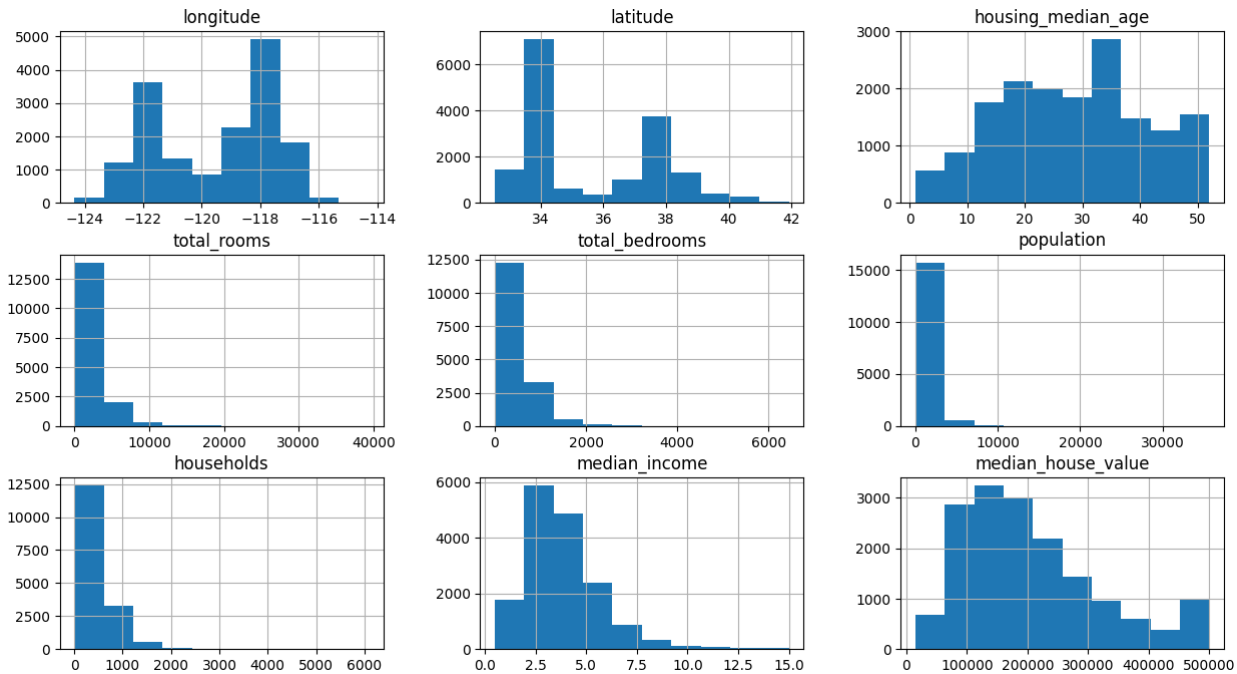
```
       longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
18245    -122.08     37.39                  46         1115
248.0
19347    -122.82     38.61                  41         2720
501.0
6363     -117.98     34.13                  29         2110
460.0
612      -122.14     37.74                  52         1071
201.0
4663     -118.31     34.05                  42          443
223.0
...          ...       ...                 ...          ...
...
6187     -117.91     34.10                  35         2746
478.0
6108     -117.88     34.13                  25         2559
654.0
17593    -121.91     37.30                  31         2095
427.0
2621     -124.05     40.94                  14         1452
```

```
217.0
11921    -117.41     33.96                  27         2341
418.0

       population  households  median_income ocean_proximity  \
18245         543         248         3.2083        NEAR BAY
19347         987         364         4.0294       <1H OCEAN
6363         1890         448         3.6806          INLAND
612           440         192         4.0662        NEAR BAY
4663          582         223         2.2937       <1H OCEAN
...           ...         ...            ...             ...
6187         1779         501         4.2500       <1H OCEAN
6108         1674         623         2.8547       <1H OCEAN
17593         829         405         3.6563       <1H OCEAN
2621          516         181         5.0329      NEAR OCEAN
11921        1272         415         3.0208          INLAND

       median_house_value
18245              334300
19347              201700
6363               130500
612                204200
4663               350000
...                   ...
6187               166700
6108               155600
17593              344700
2621               165600
11921              112700

[16346 rows x 10 columns]
```

Co-relation of the target value

```
train_data.hist(figsize=(15,8))
```

```
array([[<Axes: title={'center': 'longitude'}>,
        <Axes: title={'center': 'latitude'}>,
        <Axes: title={'center': 'housing_median_age'}>],
       [<Axes: title={'center': 'total_rooms'}>,
        <Axes: title={'center': 'total_bedrooms'}>,
        <Axes: title={'center': 'population'}>],
       [<Axes: title={'center': 'households'}>,
        <Axes: title={'center': 'median_income'}>,
        <Axes: title={'center': 'median_house_value'}>]],
dtype=object)
```



Co-relation matrix(every feature has a co-relation with each other)

Every featur has co-relation 1 with itself

```
train_data.corr()
```

```
<ipython-input-117-8d2cc80a8830>:1: FutureWarning: The default value
of numeric_only in DataFrame.corr is deprecated. In a future version,
it will default to False. Select only valid columns or specify the
value of numeric_only to silence this warning.
  train_data.corr()
```

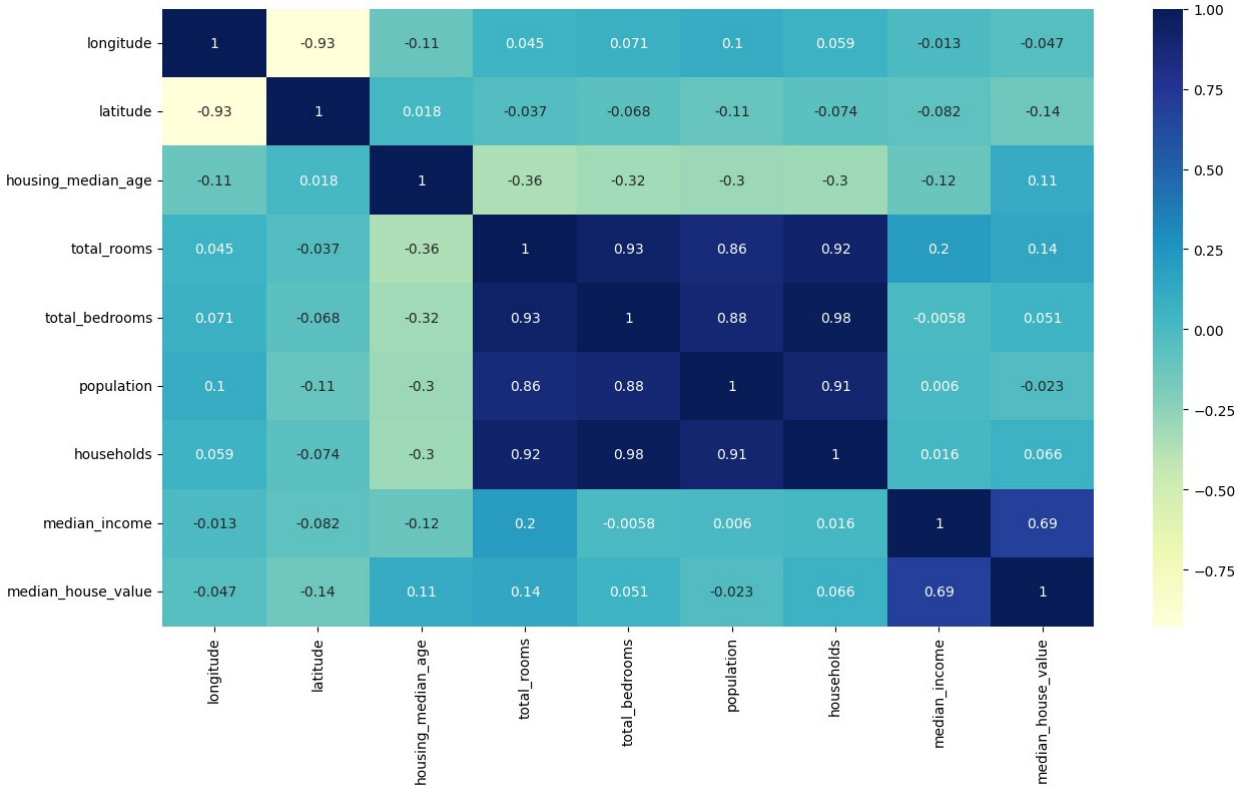
```
                     longitude  latitude  housing_median_age
total_rooms  \
longitude             1.000000 -0.925262           -0.113355
0.045294
latitude             -0.925262  1.000000            0.018129    -
0.036815
housing_median_age   -0.113355  0.018129            1.000000    -
0.358665
total_rooms           0.045294 -0.036815           -0.358665
1.000000
total_bedrooms        0.071209 -0.068434           -0.320875
0.930502
population            0.101445 -0.109371           -0.296950
```

```
0.857586
households            0.058594 -0.073644            -0.303349
0.918770
median_income        -0.013439 -0.082221            -0.115568
0.200703
median_house_value  -0.046945 -0.142268             0.107771
0.135506

                     total_bedrooms  population  households
median_income  \
longitude                  0.071209    0.101445    0.058594      -
0.013439
latitude                  -0.068434   -0.109371   -0.073644      -
0.082221
housing_median_age        -0.320875   -0.296950   -0.303349      -
0.115568
total_rooms                0.930502    0.857586    0.918770
0.200703
total_bedrooms             1.000000    0.880315    0.980009      -
0.005815
population                 0.880315    1.000000    0.909040
0.005981
households                 0.980009    0.909040    1.000000
0.015906
median_income             -0.005815    0.005981    0.015906
1.000000
median_house_value         0.051024   -0.023366    0.065846
0.686839

                     median_house_value
longitude                      -0.046945
latitude                       -0.142268
housing_median_age              0.107771
total_rooms                     0.135506
total_bedrooms                  0.051024
population                     -0.023366
households                      0.065846
median_income                   0.686839
median_house_value              1.000000
```

Create a heatmap(two dimension graphical representaton with colour) of data where the individual values that are contained ina matrix.

```
plt.figure(figsize=(15,8))
sns.heatmap(train_data.corr(), annot=True, cmap="YlGnBu")
```

```
<ipython-input-118-9ab4983852c1>:2: FutureWarning: The default value
of numeric_only in DataFrame.corr is deprecated. In a future version,
it will default to False. Select only valid columns or specify the
value of numeric_only to silence this warning.
  sns.heatmap(train_data.corr(), annot=True, cmap="YlGnBu")

<Axes: >
```

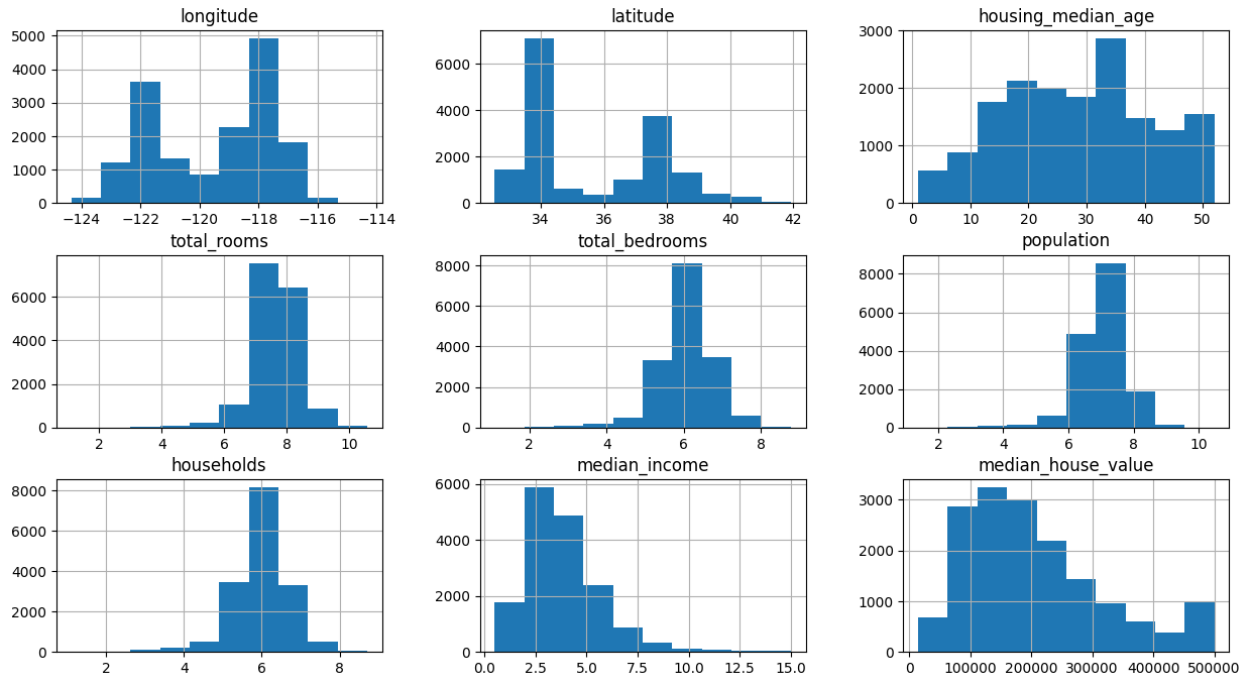


DATA PROCESSING

Logaritham of those fetures by using graph

```
train_data['total_rooms']=np.log(train_data['total_rooms'] +1)
train_data['total_bedrooms']=np.log(train_data['total_bedrooms'] +1)
train_data['population']=np.log(train_data['population'] +1)
train_data['households']=np.log(train_data['households'] +1)

train_data.hist(figsize=(15,8))

array([[<Axes: title={'center': 'longitude'}>,
        <Axes: title={'center': 'latitude'}>,
        <Axes: title={'center': 'housing_median_age'}>],
```

```
       [<Axes: title={'center': 'total_rooms'}>,
        <Axes: title={'center': 'total_bedrooms'}>,
        <Axes: title={'center': 'population'}>],
       [<Axes: title={'center': 'households'}>,
        <Axes: title={'center': 'median_income'}>,
        <Axes: title={'center': 'median_house_value'}>]],
dtype=object)
```



To get the no of houses near by <1H OCEAN, INLAND, NEAR OCEN & BAY, ISLAND (from the ocean_proximity coloumn)

```
train_data.ocean_proximity.value_counts()
```

```
<1H OCEAN     7263
INLAND        5147
NEAR OCEAN    2102
NEAR BAY      1830
ISLAND           4
Name: ocean_proximity, dtype: int64
```

We save this data into Train_data1

```
 train_data1=pd.get_dummies(train_data.ocean_proximity)

train_data1
```

```
       <1H OCEAN  INLAND  ISLAND  NEAR BAY  NEAR OCEAN
18245          0       0       0         1           0
```

```
19347          1       0       0         0           0
6363           0       1       0         0           0
612            0       0       0         1           0
4663           1       0       0         0           0
...          ...     ...     ...       ...         ...
6187           1       0       0         0           0
6108           1       0       0         0           0
17593          1       0       0         0           0
2621           0       0       0         0           1
11921          0       1       0         0           0

[16346 rows x 5 columns]
```

```
train_data
```

```
       longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
18245    -122.08     37.39                  46     7.017506
5.517453
19347    -122.82     38.61                  41     7.908755
6.218600
6363     -117.98     34.13                  29     7.654917
6.133398
612      -122.14     37.74                  52     6.977281
5.308268
4663     -118.31     34.05                  42     6.095825
5.411646
...          ...       ...                 ...          ...
...
6187     -117.91     34.10                  35     7.918265
6.171701
6108     -117.88     34.13                  25     7.847763
6.484635
17593    -121.91     37.30                  31     7.647786
6.059123
2621     -124.05     40.94                  14     7.281386
5.384495
11921    -117.41     33.96                  27     7.758761
6.037871

       population  households  median_income ocean_proximity  \
18245    6.298949    5.517453         3.2083        NEAR BAY
19347    6.895683    5.899897         4.0294       <1H OCEAN
6363     7.544861    6.107023         3.6806          INLAND
612      6.089045    5.262690         4.0662        NEAR BAY
4663     6.368187    5.411646         2.2937       <1H OCEAN
...           ...         ...            ...             ...
6187     7.484369    6.218600         4.2500       <1H OCEAN
6108     7.423568    6.436150         2.8547       <1H OCEAN
17593    6.721426    6.006353         3.6563       <1H OCEAN
```

```
2621     6.248043    5.204007          5.0329      NEAR OCEAN
11921    7.149132    6.030685          3.0208          INLAND

       median_house_value
18245              334300
19347              201700
6363               130500
612                204200
4663               350000
...                   ...
6187               166700
6108               155600
17593              344700
2621               165600
11921              112700

[16346 rows x 10 columns]
```

Then we join train_data with trai_data1 which is store into the train_data

```
train_data=train_data.join(train_data1)

train_data
```

```
       longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
18245    -122.08     37.39                  46     7.017506
5.517453
19347    -122.82     38.61                  41     7.908755
6.218600
6363     -117.98     34.13                  29     7.654917
6.133398
612      -122.14     37.74                  52     6.977281
5.308268
4663     -118.31     34.05                  42     6.095825
5.411646
...          ...       ...                 ...          ...
...
6187     -117.91     34.10                  35     7.918265
6.171701
6108     -117.88     34.13                  25     7.847763
6.484635
17593    -121.91     37.30                  31     7.647786
6.059123
2621     -124.05     40.94                  14     7.281386
5.384495
11921    -117.41     33.96                  27     7.758761
6.037871

       population  households  median_income ocean_proximity  \
```

```
18245    6.298949    5.517453         3.2083        NEAR BAY
19347    6.895683    5.899897         4.0294       <1H OCEAN
6363     7.544861    6.107023         3.6806          INLAND
612      6.089045    5.262690         4.0662        NEAR BAY
4663     6.368187    5.411646         2.2937       <1H OCEAN
...           ...         ...            ...             ...
6187     7.484369    6.218600         4.2500       <1H OCEAN
6108     7.423568    6.436150         2.8547       <1H OCEAN
17593    6.721426    6.006353         3.6563       <1H OCEAN
2621     6.248043    5.204007         5.0329      NEAR OCEAN
11921    7.149132    6.030685         3.0208          INLAND

       median_house_value  <1H OCEAN  INLAND  ISLAND  NEAR BAY  NEAR
OCEAN
18245              334300          0       0       0         1
0
19347              201700          1       0       0         0
0
6363               130500          0       1       0         0
0
612                204200          0       0       0         1
0
4663               350000          1       0       0         0
0
...                   ...        ...     ...     ...       ...
...
6187               166700          1       0       0         0
0
6108               155600          1       0       0         0
0
17593              344700          1       0       0         0
0
2621               165600          0       0       0         0
1
11921              112700          0       1       0         0
0

[16346 rows x 15 columns]
```

Then we drop the "ocean_proximity" because each value of "ocean_proximity" has declare separately

```
train_data=train_data.drop(['ocean_proximity'],axis=1)

train_data
```

```
       longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
18245    -122.08     37.39                  46     7.017506
5.517453
```

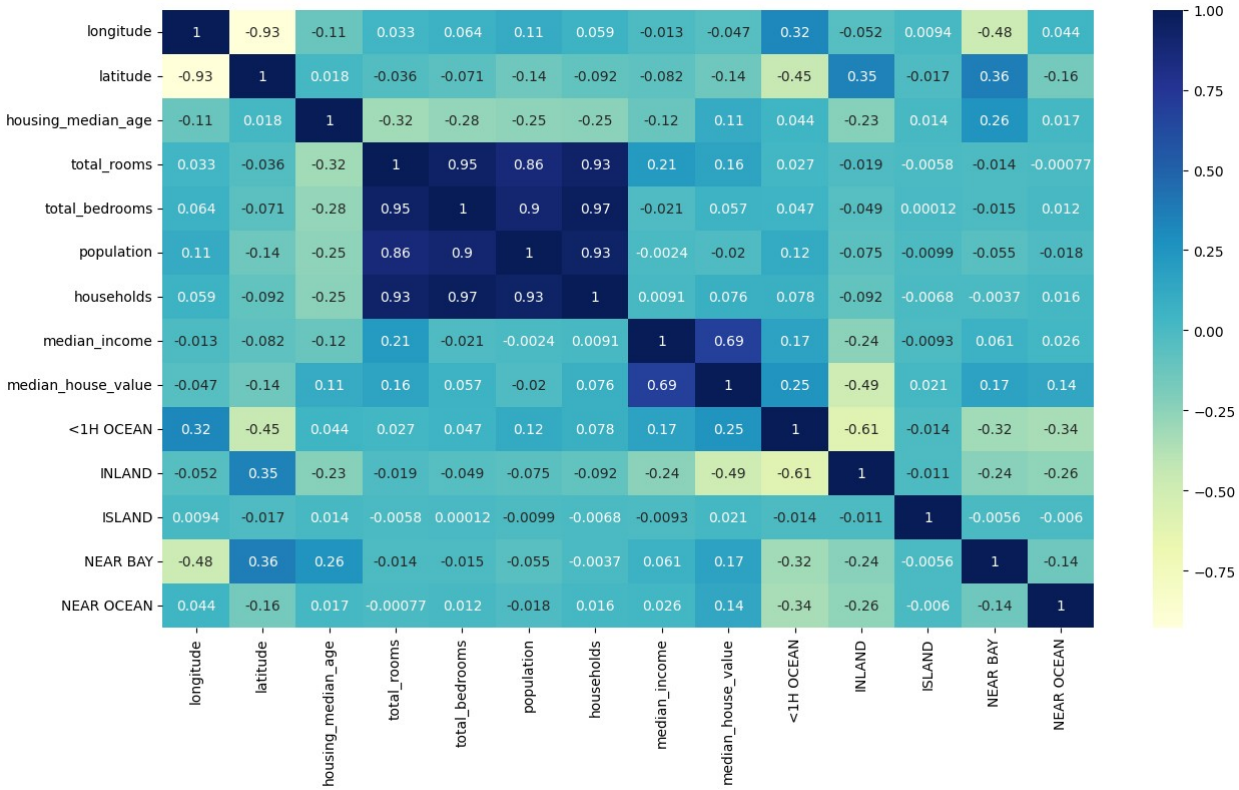
```
19347    -122.82     38.61                  41     7.908755
6.218600
6363     -117.98     34.13                  29     7.654917
6.133398
612      -122.14     37.74                  52     6.977281
5.308268
4663     -118.31     34.05                  42     6.095825
5.411646
...          ...       ...                 ...          ...
...
6187     -117.91     34.10                  35     7.918265
6.171701
6108     -117.88     34.13                  25     7.847763
6.484635
17593    -121.91     37.30                  31     7.647786
6.059123
2621     -124.05     40.94                  14     7.281386
5.384495
11921    -117.41     33.96                  27     7.758761
6.037871

       population  households  median_income  median_house_value  <1H
OCEAN  \
18245    6.298949    5.517453         3.2083              334300
0
19347    6.895683    5.899897         4.0294              201700
1
6363     7.544861    6.107023         3.6806              130500
0
612      6.089045    5.262690         4.0662              204200
0
4663     6.368187    5.411646         2.2937              350000
1
...           ...         ...            ...                 ...
...
6187     7.484369    6.218600         4.2500              166700
1
6108     7.423568    6.436150         2.8547              155600
1
17593    6.721426    6.006353         3.6563              344700
1
2621     6.248043    5.204007         5.0329              165600
0
11921    7.149132    6.030685         3.0208              112700
0

       INLAND  ISLAND  NEAR BAY  NEAR OCEAN  bedroom_ratio
household_rooms
18245       0       0         1           0       0.786241
```

```
1.271874
19347       0       0         0           0       0.786293
1.340490
6363        1       0         0           0       0.801236
1.253461
612         0       0         1           0       0.760793
1.325801
4663        0       0         0           0       0.887763
1.126427
...       ...     ...       ...         ...            ...
...
6187        0       0         0           0       0.779426
1.273319
6108        0       0         0           0       0.826304
1.219326
17593       0       0         0           0       0.792272
1.273283
2621        0       0         0           1       0.739488
1.399188
11921       1       0         0           0       0.778200
1.286547

[16346 rows x 16 columns]
```

After that we show the heatmap of the train_data

```
plt.figure(figsize=(15,8))
sns.heatmap(train_data.corr(), annot=True, cmap="YlGnBu")
```

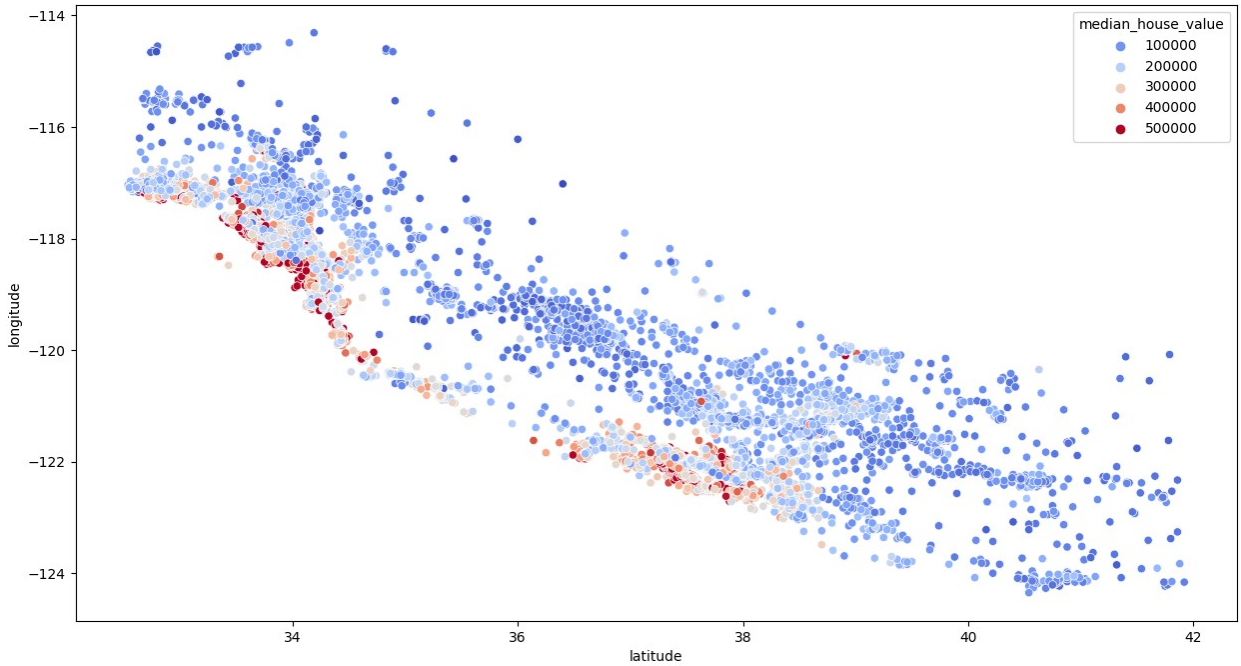
```
<ipython-input-128-9ab4983852c1>:2: FutureWarning: The default value
of numeric_only in DataFrame.corr is deprecated. In a future version,
it will default to False. Select only valid columns or specify the
value of numeric_only to silence this warning.
  sns.heatmap(train_data.corr(), annot=True, cmap="YlGnBu")

<Axes: >
```



Visualization co-ordinate with plot function

```
plt.figure(figsize=(15,8))
sns.scatterplot(x="latitude",y="longitude",data=train_data,hue="median
_house_value", palette="coolwarm")
```

```
<Axes: xlabel='latitude', ylabel='longitude'>
```



FEATURE ENGINEERING

We calculate the ration of bedrooms and household_rooms

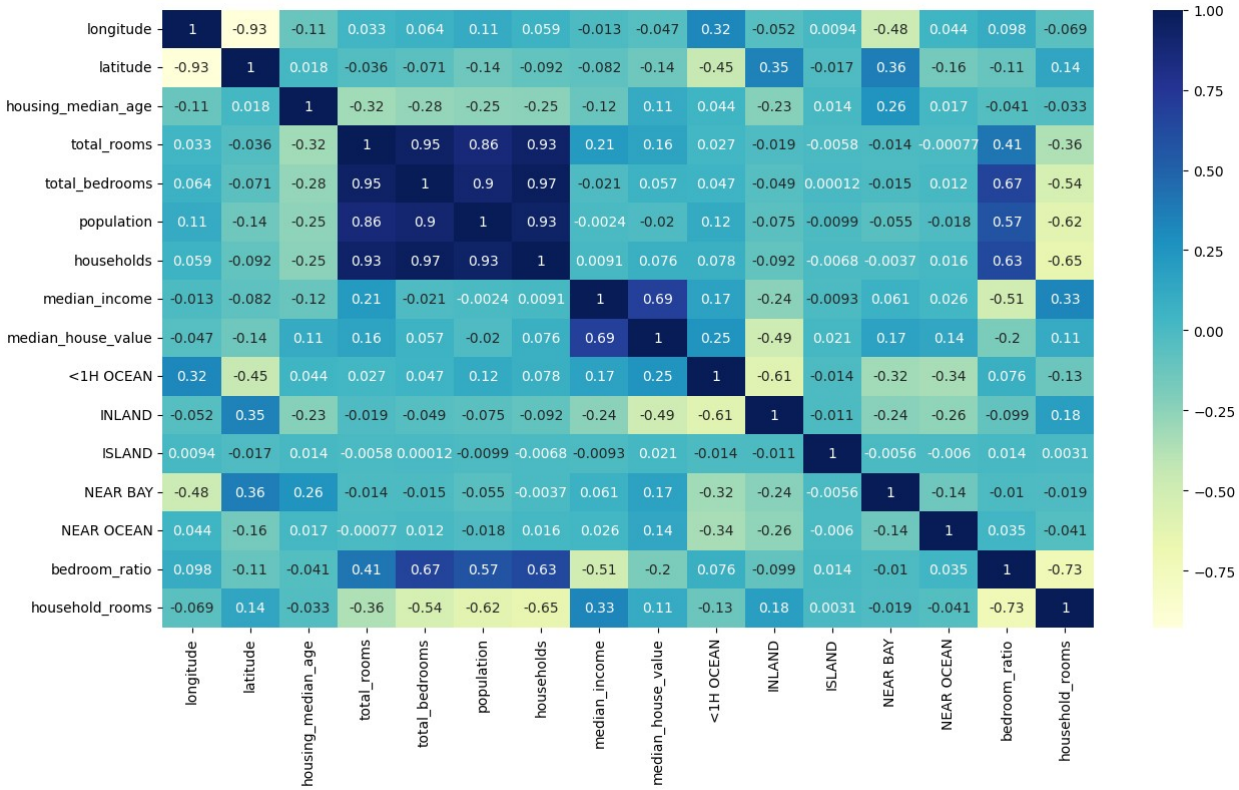
```
train_data['bedroom_ratio'] =
train_data['total_bedrooms']/train_data['total_rooms']
train_data['household_rooms']=
train_data['total_rooms']/train_data['households']
```

Now we show a heatmap to show a co-relation of 'bedroom_ratio','household_rooms' with respect to 'median_house_value'

```
plt.figure(figsize=(15,8))
sns.heatmap(train_data.corr(), annot=True, cmap="YlGnBu")
```

```
<ipython-input-131-9ab4983852c1>:2: FutureWarning: The default value
of numeric_only in DataFrame.corr is deprecated. In a future version,
it will default to False. Select only valid columns or specify the
value of numeric_only to silence this warning.
  sns.heatmap(train_data.corr(), annot=True, cmap="YlGnBu")
```

```
<Axes: >
```



LINEAR REGRATION MODEL

```
  from sklearn.linear_model import LinearRegression

model = linear_model.LinearRegression()

model

LinearRegression()

x_train,y_train=train_data.drop(['median_house_value'],axis=1),
train_data['median_house_value']

from sklearn.linear_model import LinearRegression

x_train, y_train = train_data.drop(['median_house_value'], axis=1),
train_data['median_house_value']

reg = LinearRegression()
```

```
reg.fit(x_train, y_train)
```

```
LinearRegression()
```

```
test_data=x_test.join(y_test)
test_data['total_rooms']=np.log(test_data['total_rooms'] +1)
test_data['total_bedrooms']=np.log(test_data['total_bedrooms'] +1)
test_data['population']=np.log(test_data['population'] +1)
test_data['households']=np.log(test_data['households'] +1)

test_data=test_data.join(pd.get_dummies(test_data.ocean_proximity)).dr
op(['ocean_proximity'],axis=1)

test_data['bedroom_ratio'] =
test_data['total_bedrooms']/test_data['total_rooms']
test_data['household_rooms']=
test_data['total_rooms']/test_data['households']

x_test,y_test=test_data.drop(['median_house_value'],axis=1),
test_data['median_house_value']

x_test
```

```
       longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
2327     -119.71     36.81                  19     7.407924
5.910797
16426    -121.42     38.22                  35     7.318540
5.749393
3094     -118.45     35.58                  16     8.593599
7.075809
14421    -117.23     32.80                  22     8.000349
6.773080
15631    -122.40     37.80                  52     7.404279
6.347389
...          ...       ...                 ...          ...
...
11817    -121.05     38.97                  12     8.209852
6.311735
267      -122.19     37.77                  52     7.753624
6.100319
6736     -118.10     34.13                  47     7.711997
5.624018
18663    -121.94     37.00                  32     7.701200
6.056784
```

```
1549     -121.92     37.73                  24     7.249926
5.793014

       population  households  median_income  <1H OCEAN  INLAND
ISLAND  \
2327     6.324359    5.872118         1.7969          0       1
0
16426    6.767343    5.648974         2.0284          0       1
0
3094     7.497207    6.694562         1.8819          0       1
0
14421    7.468513    6.614726         2.3482          0       0
0
15631    7.267525    6.242223         1.9063          0       0
0
...           ...         ...            ...        ...     ...     ..
.
11817    7.360740    6.236370         4.8214          0       1
0
267      7.043160    6.035481         3.5114          0       0
0
6736     6.620073    5.564520        15.0001          1       0
0
18663    6.987490    5.983936         4.1587          0       0
0
1549     6.218600    5.690359         2.4821          1       0
0

       NEAR BAY  NEAR OCEAN  bedroom_ratio  household_rooms
2327          0           0       0.797902         1.261542
16426         0           0       0.785593         1.295552
3094          0           0       0.823381         1.283669
14421         0           1       0.846598         1.209476
15631         1           0       0.857260         1.186161
...         ...         ...            ...              ...
11817         0           0       0.768800         1.316447
267           1           0       0.786770         1.284674
6736          0           0       0.729256         1.385923
18663         0           1       0.786473         1.286979
1549          0           0       0.799045         1.274072

[4087 rows x 15 columns]

x_train

       longitude  latitude  housing_median_age  total_rooms
total_bedrooms  \
18245    -122.08     37.39                  46     7.017506
5.517453
19347    -122.82     38.61                  41     7.908755
```

```
6.218600
6363     -117.98     34.13                  29     7.654917
6.133398
612      -122.14     37.74                  52     6.977281
5.308268
4663     -118.31     34.05                  42     6.095825
5.411646
...          ...       ...                 ...          ...
...
6187     -117.91     34.10                  35     7.918265
6.171701
6108     -117.88     34.13                  25     7.847763
6.484635
17593    -121.91     37.30                  31     7.647786
6.059123
2621     -124.05     40.94                  14     7.281386
5.384495
11921    -117.41     33.96                  27     7.758761
6.037871

       population  households  median_income  <1H OCEAN  INLAND
ISLAND  \
18245    6.298949    5.517453         3.2083          0       0
0
19347    6.895683    5.899897         4.0294          1       0
0
6363     7.544861    6.107023         3.6806          0       1
0
612      6.089045    5.262690         4.0662          0       0
0
4663     6.368187    5.411646         2.2937          1       0
0
...           ...         ...            ...        ...     ...      ..
.
6187     7.484369    6.218600         4.2500          1       0
0
6108     7.423568    6.436150         2.8547          1       0
0
17593    6.721426    6.006353         3.6563          1       0
0
2621     6.248043    5.204007         5.0329          0       0
0
11921    7.149132    6.030685         3.0208          0       1
0

       NEAR BAY  NEAR OCEAN  bedroom_ratio  household_rooms
18245         1           0       0.786241         1.271874
19347         0           0       0.786293         1.340490
6363          0           0       0.801236         1.253461
```

```
612           1           0       0.760793         1.325801
4663          0           0       0.887763         1.126427
...         ...         ...            ...              ...
6187          0           0       0.779426         1.273319
6108          0           0       0.826304         1.219326
17593         0           0       0.792272         1.273283
2621          0           1       0.739488         1.399188
11921         0           0       0.778200         1.286547

[16346 rows x 15 columns]
```

```
reg.score(x_test,y_test)
```

```
0.6782240713244606
```

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
scaler= StandardScaler()
x_train, y_train = train_data.drop(['median_house_value'], axis=1),
train_data['median_house_value']
x_train_s = scaler.fit_transform(x_train)

reg = LinearRegression()
reg.fit(x_train_s, y_train)
```

```
LinearRegression()
```

```
test_data=x_test.join(y_test)
test_data['total_rooms']=np.log(test_data['total_rooms'] +1)
test_data['total_bedrooms']=np.log(test_data['total_bedrooms'] +1)
test_data['population']=np.log(test_data['population'] +1)
test_data['households']=np.log(test_data['households'] +1)

test_data=test_data.join(pd.get_dummies(test_data.ocean_proximity)).dr
op(['ocean_proximity'],axis=1)

test_data['bedroom_ratio'] =
test_data['total_bedrooms']/test_data['total_rooms']
test_data['household_rooms']=
test_data['total_rooms']/test_data['households']
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call
last)
<ipython-input-152-8f34d0525a90> in <cell line: 7>()
      5 test_data['households']=np.log(test_data['households'] +1)
      6
----> 7
test_data=test_data.join(pd.get_dummies(test_data.ocean_proximity)).dr
op(['ocean_proximity'],axis=1)
```

```
      8
      9 test_data['bedroom_ratio'] =
test_data['total_bedrooms']/test_data['total_rooms']

/usr/local/lib/python3.10/dist-packages/pandas/core/generic.py in
__getattr__(self, name)
   5900         ):
   5901             return self[name]
-> 5902         return object.__getattribute__(self, name)
   5903
   5904     def __setattr__(self, name: str, value) -> None:

AttributeError: 'DataFrame' object has no attribute 'ocean_proximity'
```

```
x_test,y_test=test_data.drop(['median_house_value'],axis=1),
test_data['median_house_value']
```

```
x_test_s = scaler.transform(x_test)
```

```
reg.score(x_test_s,y_test)
```

```
-0.5407857077050962
```

RANDOM FOREST MODEL

```
from sklearn.ensemble import RandomForestRegressor
```

```
forest = RandomForestRegressor()
```

```
forest.fit(x_train, y_train)
```

```
RandomForestRegressor()
```

```
forest.score(x_test, y_test)
```

```
0.4774857816871597
```

Cross validatiuon

```
from sklearn.ensemble import RandomForestRegressor
```

```
forest = RandomForestRegressor()
```

```
forest.fit(x_train_s, y_train)
```

```
RandomForestRegressor()
```

```
forest.score(x_test_s,y_test)
```

```
0.46855162665922223
```

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "n_estimators": [3,10,30],
    "max_features": [2,4,6,8]
}

grid_search = GridSearchCV(forest, param_grid, cv=5,
                           scoring="neg_mean_squared_error",
                           return_train_score=True)

grid_search.fit(x_train_s, y_train)
```

```
GridSearchCV(cv=5, estimator=RandomForestRegressor(),
             param_grid={'max_features': [2, 4, 6, 8],
                         'n_estimators': [3, 10, 30]},
             return_train_score=True,
scoring='neg_mean_squared_error')
```

```
from sklearn.model_selection import GridSearchCV

forest = RandomForestRegressor()

param_grid = {
    "n_estimators": [3,10,30],
    "max_features": [2,4,6,8]
}

grid_search = GridSearchCV(forest, param_grid, cv=5,
                           scoring="neg_mean_squared_error",
                           return_train_score=True)

grid_search.fit(x_train_s, y_train)
```

```
GridSearchCV(cv=5, estimator=RandomForestRegressor(),
             param_grid={'max_features': [2, 4, 6, 8],
                         'n_estimators': [3, 10, 30]},
             return_train_score=True,
scoring='neg_mean_squared_error')
```

```
grid_search.best_estimator_
```

```
RandomForestRegressor(max_features=8, n_estimators=30)
```

```
best_forest = grid_search.best_estimator_

best_forest.score(x_test_s,y_test)
```

```
0.5039987276661204
```